HomeGuard Security System Lab - Complete Beginner Breakdown

Table of Contents

1. Before You Start: Key Concepts
2. Step 1: Project Setup
3. Step 2: Data Structures
4. Step 3: If/Else Logic
5. Step 4: Functions
6. Step 5: Classes
7. Step 6: Integration
8. Step 7: Testing

BEFORE YOU START: Key Concepts for This Lab 🎓

What is the HomeGuard System?

Think of HomeGuard as a **robot that watches your house** while you're away. It has sensors (like eyes and ears) that detect things, and it makes decisions about whether to alert you.

The Main Concepts You'll Use:

1. Dictionaries (Data Containers)

A dictionary is like a **notebook** that stores information with labels.

```python
# Example: A dictionary for ONE sensor
sensor = {
    "id": "MOTION_001",        # Key: "id", Value: "MOTION_001"
    "location": "Living Room", # Key: "location", Value: "Living Room"
    "type": "motion",          # Key: "type", Value: "motion"
    "current_value": None      # Key: "current_value", Value: None
}
```

Why? This keeps all information about ONE sensor together. Instead of having 4 separate variables, you have 1 dictionary with 4 labeled pieces of information.

2. Lists (Collections of Dictionaries)

A list is like a **folder that holds multiple notebooks**.

```python
# Example: A list holding MULTIPLE sensors
sensors = [
    {"id": "MOTION_001", "location": "Living Room", "type": "motion"},
    {"id": "TEMP_001", "location": "Kitchen", "type": "temperature"},
    {"id": "DOOR_001", "location": "Front Door", "type": "door"}
]
```

3. If/Else Statements (Decision Making)

These are like **rules your robot follows**:

- IF the temperature is below 35°F, THEN trigger a "CRITICAL" alert
- ELSE (if it's not below 35°F), don't trigger an alert

4. Functions (Reusable Instructions)

A function is like a **recipe card** that you can follow multiple times:

- Input: ingredients you provide
- Process: the steps the function performs
- Output: the final result

5. Classes (Blueprints)

A class is like a **template** for creating sensor objects. Instead of manually creating each sensor dictionary, a class creates them automatically with all the right information.

STEP 1: PROJECT SETUP

What You're Doing

Creating a Python file that sets up the basic structure. Think of it like building the **foundation of a house**.

The Code Explained

```python
"""
HomeGuard Security System Simulator
Author: Your Name
Description: A smart home monitoring system that processes sensor readings
             and triggers alerts for security, safety, and comfort issues.
"""
```

Explanation:

- These three quotation marks `"""` create a **multi-line comment**
- Comments are notes that Python ignores (they don't run as code)
- This tells anyone reading your code what the project does

```python
import random
from datetime import datetime
```

Explanation:

- `import random` - brings in Python's randomness tool. You'll use `random.randint()` and `random.choice()` to create fake sensor readings
- `from datetime import datetime` - brings in time/date tools. You'll use this to add timestamps like "14:30:00"

```python
# System configuration
HOME_MODES = ["HOME", "AWAY", "SLEEP"]
ALERT_SEVERITIES = ["LOW", "MEDIUM", "HIGH", "CRITICAL"]

# Current system state
current_mode = "AWAY"
```

Explanation:

- `HOME_MODES` is a **list** (with square brackets) that stores the three possible states of your house
- `ALERT_SEVERITIES` is a **list** that stores how serious an alert is
- `current_mode` is a **variable** that stores the current state. It's set to "AWAY" meaning the house is empty

Checkpoint: Run This Code

Create a new Jupyter notebook called `homeguard_system.ipynb` and paste the code above. Run it. You should see **no output** because we're just setting things up. That's correct!

STEP 2: DATA STRUCTURES

What You're Doing

Creating the **containers** (dictionaries) that will hold information about sensors and alerts.

What is a Dictionary?

Think of a dictionary like a **filing cabinet drawer**:

- The **key** is the label on the folder (like "id", "location", "type")
- The **value** is the information inside (like "MOTION_001", "Living Room", "motion")

```python
# Here's one sensor as a dictionary:
sensor = {
    "id": "MOTION_001",
    "location": "Living Room",
    "type": "motion",
    "current_value": None,
    "threshold": None
}
```

Reading a Dictionary

To **get information** from a dictionary, use square brackets:

```python
sensor["id"]        # Returns: "MOTION_001"
sensor["location"]  # Returns: "Living Room"
sensor["type"]      # Returns: "motion"
```

To **change information** in a dictionary:

```python
sensor["current_value"] = True  # Changes current_value from None to True
```

The Function: create_sensor()

A function is like a **recipe**. Instead of writing the same dictionary over and over, you create a function that builds it for you.

```python
```

```python
def create_sensor(sensor_id, location, sensor_type, threshold=None):
    """
    Creates a sensor data structure.

    Parameters:
    - sensor_id: Unique identifier for the sensor (like "MOTION_001")
    - location: Where the sensor is (like "Living Room")
    - sensor_type: What kind of sensor ("motion", "temperature", "door", "smoke")
    - threshold: Optional limit value (default is None)

    Returns:
    - A dictionary representing the sensor
    """
    sensor = {
        "id": sensor_id,          # Store the id you passed in
        "location": location,     # Store the location you passed in
        "type": sensor_type,      # Store the type you passed in
        "current_value": None,    # No reading yet, so this is None
        "threshold": threshold    # Store the threshold (if provided)
    }
    return sensor  # Give back the completed dictionary
```

Step-by-Step Breakdown of create_sensor()

Line 1: `def create_sensor(sensor_id, location, sensor_type, threshold=None):`

- `def` means "define a function"
- `sensor_id, location, sensor_type, threshold` are **parameters** (information you provide when calling the function)
- `threshold=None` means this parameter is **optional**. If you don't provide it, it defaults to `None`

Lines 2-7: The docstring (explanation of what the function does)

- This is for humans reading the code, not for Python

Lines 8-14: Create the dictionary

- We create a variable called `sensor` that is a dictionary
- We fill it with the information passed in

Line 15: `return sensor`

- The function **gives back** the completed dictionary
- Whatever called this function now has the sensor dictionary to use

How to Use create_sensor()

```python
# Creating a motion sensor
sensor1 = create_sensor("MOTION_001", "Living Room", "motion")

# Creating a temperature sensor with a threshold
sensor2 = create_sensor("TEMP_001", "Kitchen", "temperature", threshold=35)

# What's inside sensor1?
print(sensor1)
# Output: {'id': 'MOTION_001', 'location': 'Living Room', 'type': 'motion', 'current_value': None, 'threshold': None}
```

The Second Function: create_alert()

This function creates the **alert dictionaries** (the warnings your system triggers).

```python
def create_alert(severity, message, sensor_id, timestamp):
    """
    Creates an alert data structure.

    Parameters:
    - severity: How serious ("LOW", "MEDIUM", "HIGH", "CRITICAL")
    - message: Description of the alert (like "Motion detected in living room")
    - sensor_id: Which sensor triggered this alert
    - timestamp: When the alert happened (like "14:30:00")

    Returns:
    - A dictionary representing the alert
    """
    alert = {
        "severity": severity,        # Store severity level
        "message": message,          # Store the alert message
        "sensor_id": sensor_id,      # Store which sensor caused this
        "timestamp": timestamp       # Store when it happened
    }
    return alert  # Give back the alert dictionary
```

How to Use create_alert()

```python
```

```python
# Creating an alert
alert1 = create_alert("HIGH", "Motion detected in Living Room while in AWAY mode!", "MOTION_001", "14:30:00")

print(alert1)
# Output: {'severity': 'HIGH', 'message': 'Motion detected in Living Room while in AWAY mode!', 'sensor_id': 'MOTION_001'
```

Creating All Sensors

Now that you have the `create_sensor()` function, use it to create all 4 sensors:

```python
# Initialize sensors for the Peterson home
sensors = [
    create_sensor("MOTION_001", "Living Room", "motion"),
    create_sensor("TEMP_001", "Kitchen", "temperature", threshold=35),
    create_sensor("DOOR_001", "Front Door", "door"),
    create_sensor("SMOKE_001", "Bedroom", "smoke")
]
```

Understanding This Code

Line 1: `sensors = [`

- Start creating a list called `sensors`

Lines 2-5: Four function calls

- Each line calls `create_sensor()` and returns a dictionary
- We're putting each dictionary inside the list

Line 6: `]`

- End the list

What's Inside?

```python
sensors[0]  # The motion sensor dictionary
sensors[1]  # The temperature sensor dictionary
sensors[2]  # The door sensor dictionary
sensors[3]  # The smoke sensor dictionary
```

Checkpoint: Test Your Code

Add this print statement to verify everything works:

```python
print(f"Initialized {len(sensors)} sensors")
for sensor in sensors:
    print(f"  - {sensor['id']}: {sensor['location']} ({sensor['type']})")
```

Step-by-Step Explanation:

Line 1: `print(f"Initialized {len(sensors)} sensors")`

- `len(sensors)` counts how many sensors are in the list (should be 4)
- `f"..."` is an f-string that inserts the count
- Output: `Initialized 4 sensors`

Line 2: `for sensor in sensors:`

- Loop through each sensor in the sensors list
- Each time, `sensor` becomes one sensor dictionary

Line 3: `print(f"  - {sensor['id']}: {sensor['location']} ({sensor['type']})")`

- Print the sensor's id, location, and type
- `sensor['id']` gets the value from the "id" key
- Output looks like: `  - MOTION_001: Living Room (motion)`

Expected Output:

```
Initialized 4 sensors
  - MOTION_001: Living Room (motion)
  - TEMP_001: Kitchen (temperature)
  - DOOR_001: Front Door (door)
  - SMOKE_001: Bedroom (smoke)
```

STEP 3: IF/ELSE LOGIC

What You're Doing

Creating **decision-making** functions that check if sensor readings are abnormal or dangerous.

Understanding If/Else

An if/else statement is like a **decision tree**:

```
IF condition is true:
   do this thing
ELSE:
   do this other thing
```

Real-World Example

```
IF temperature < 35:
   trigger "CRITICAL: Frozen pipes risk!" alert
ELSE:
   no alert needed
```

Function 1: is_abnormal_reading()

This function checks if a sensor's reading is **weird or dangerous**.

```python
```

```python
def is_abnormal_reading(sensor, reading_value):
    """
    Checks if a sensor reading is abnormal based on sensor type and thresholds.

    Parameters:
    - sensor: Sensor dictionary
    - reading_value: The current reading from the sensor

    Returns:
    - True if abnormal, False if normal
    """
    sensor_type = sensor["type"]

    # Check temperature sensors
    if sensor_type == "temperature":
        if reading_value < 35 or reading_value > 95:
            return True  # Abnormal!
        else:
            return False  # Normal

    # Check motion sensors
    elif sensor_type == "motion":
        if reading_value == True:
            return True  # Motion detected = abnormal for monitoring
        else:
            return False  # No motion = normal

    # Check door sensors
    elif sensor_type == "door":
        if reading_value == "OPEN":
            return True  # Door open = abnormal
        else:
            return False  # Door closed = normal

    # Check smoke sensors
    elif sensor_type == "smoke":
        if reading_value == "DETECTED":
            return True  # Smoke = abnormal!
        else:
            return False  # No smoke = normal

    # If we don't recognize the sensor type
    return False
```

Step-by-Step Breakdown:

Line 1: `sensor_type = sensor["type"]`

- Get the type of sensor (like "motion", "temperature", etc.)
- Store it in a variable for easier reading

Block 1: Temperature Check

```python
if sensor_type == "temperature":
    if reading_value < 35 or reading_value > 95:
        return True
    else:
        return False
```

Explanation:

- First IF: "Is this a temperature sensor?"
- Second IF: "Is the reading below 35°F OR above 95°F?"
- `<` means "less than"
- `>` means "greater than"
- `or` means both conditions matter (if either is true, it's abnormal)
- If abnormal: `return True`
- If normal: `return False`

Block 2: Motion Check

```python
elif sensor_type == "motion":
    if reading_value == True:
        return True
    else:
        return False
```

Explanation:

- `elif` means "else if" (check this only if previous conditions were false)
- Motion sensors return True when motion is detected
- If `reading_value == True` (motion WAS detected), that's abnormal
- Return True (abnormal)

Block 3: Door Check

```python
elif sensor_type == "door":
    if reading_value == "OPEN":
        return True
    else:
        return False
```

Explanation:

- If the sensor is a door sensor
- If the door is "OPEN", that's abnormal
- Return True

Block 4: Smoke Check

```python
elif sensor_type == "smoke":
    if reading_value == "DETECTED":
        return True
    else:
        return False
```

Explanation:

- If smoke is "DETECTED", that's abnormal
- Return True

Function 2: should_trigger_security_alert()

This function checks if a **security alert** should happen (motion or door in AWAY mode).

```python
```

```python
def should_trigger_security_alert(sensor, reading_value, system_mode):
    """
    Determines if a security alert should be triggered.

    Parameters:
    - sensor: Sensor dictionary
    - reading_value: The current reading from the sensor
    - system_mode: Current system mode ("HOME", "AWAY", "SLEEP")

    Returns:
    - True if security alert should trigger, False otherwise
    """
    sensor_type = sensor["type"]

    # Security alerts only for motion and door sensors
    if sensor_type == "motion" or sensor_type == "door":
        # Only alert if the system is in AWAY mode
        if system_mode == "AWAY":
            # Check if the sensor detected something abnormal
            if is_abnormal_reading(sensor, reading_value):
                return True  # Yes, trigger security alert!
            else:
                return False  # False alarm, reading is normal
        else:
            return False  # Not in AWAY mode, no security alert
    else:
        return False  # Not a motion/door sensor, no security alert
```

Step-by-Step Breakdown:

Line 1: `sensor_type = sensor["type"]`

- Get the sensor type

Block 1:

```python
if sensor_type == "motion" or sensor_type == "door":
```

Explanation:

- Check if this is a motion sensor OR a door sensor
- If it's not one of these, we don't care about security

Block 2:

```python
if system_mode == "AWAY":
```

Explanation:

- Only trigger security alerts if the house is in AWAY mode
- If you're home, motion is normal!

Block 3:

```python
if is_abnormal_reading(sensor, reading_value):
    return True
else:
    return False
```

Explanation:

- Use the function we just created!
- Call `is_abnormal_reading()` with the sensor and reading
- If it returns True (abnormal), we trigger a security alert
- If it returns False (normal), we don't

Checkpoint: Test Your Code

```python
```

```python
# Test 1: Temperature sensor
test_sensor = create_sensor("TEMP_TEST", "Test Room", "temperature", threshold=35)

print("Temperature Tests:")
print(f"34°F is abnormal: {is_abnormal_reading(test_sensor, 34)}")  # Should print: True
print(f"68°F is abnormal: {is_abnormal_reading(test_sensor, 68)}")  # Should print: False
print(f"100°F is abnormal: {is_abnormal_reading(test_sensor, 100)}")  # Should print: True
print()

# Test 2: Motion sensor
motion_sensor = create_sensor("MOTION_TEST", "Test Room", "motion")

print("Security Alert Tests:")
print(f"Motion in AWAY mode triggers alert: {should_trigger_security_alert(motion_sensor, True, 'AWAY')}")  # True
print(f"Motion in HOME mode triggers alert: {should_trigger_security_alert(motion_sensor, True, 'HOME')}")  # False
print()

# Test 3: Door sensor
door_sensor = create_sensor("DOOR_TEST", "Front Door", "door")

print("Door Tests:")
print(f"Open door in AWAY mode triggers alert: {should_trigger_security_alert(door_sensor, 'OPEN', 'AWAY')}")  # True
print(f"Closed door in AWAY mode triggers alert: {should_trigger_security_alert(door_sensor, 'CLOSED', 'AWAY')}")  # Fal
```

Expected Output:

```
Temperature Tests:
34°F is abnormal: True
68°F is abnormal: False
100°F is abnormal: True

Security Alert Tests:
Motion in AWAY mode triggers alert: True
Motion in HOME mode triggers alert: False

Door Tests:
Open door in AWAY mode triggers alert: True
Closed door in AWAY mode triggers alert: False
```

STEP 4: FUNCTIONS FOR PROCESSING

What You're Doing

Creating functions that **generate fake sensor readings**, **process them to decide on alerts**, and **display alerts** to the user.

Function 1: generate_reading()

This function **makes up realistic sensor readings** to simulate real sensors.

```python
```

```python
def generate_reading(sensor):
    """
    Generates a realistic reading for a sensor.

    Parameters:
    - sensor: Sensor dictionary

    Returns:
    - A realistic reading value based on sensor type
    """
    sensor_type = sensor["type"]

    if sensor_type == "temperature":
        # Generate a random temperature between 30-100°F
        return random.randint(30, 100)

    elif sensor_type == "motion":
        # Randomly return True (motion detected) or False (no motion)
        # 20% chance of motion, 80% chance of no motion
        return random.random() < 0.2

    elif sensor_type == "door":
        # Randomly return "OPEN" or "CLOSED"
        # 10% chance of open, 90% chance of closed
        if random.random() < 0.1:
            return "OPEN"
        else:
            return "CLOSED"

    elif sensor_type == "smoke":
        # Randomly return "CLEAR" or "DETECTED"
        # 5% chance of smoke detected, 95% chance of clear
        if random.random() < 0.05:
            return "DETECTED"
        else:
            return "CLEAR"

    return None  # Unknown sensor type
```

Step-by-Step Breakdown:

Temperature Block:

```python
```

```
if sensor_type == "temperature":
    return random.randint(30, 100)
```

Explanation:

- `random.randint(30, 100)` picks a random whole number between 30 and 100 (inclusive)
- Returns a number like 67, 45, 89, etc.

Motion Block:

```python
elif sensor_type == "motion":
    return random.random() < 0.2
```

Explanation:

- `random.random()` generates a random number between 0.0 and 1.0
- `< 0.2` means "is this number less than 0.2?"
- Since 0.2 = 20%, this returns True 20% of the time, False 80% of the time
- So motion is detected 20% of the time (realistic for an empty house)

Door Block:

```python
elif sensor_type == "door":
    if random.random() < 0.1:
        return "OPEN"
    else:
        return "CLOSED"
```

Explanation:

- 10% chance the door is open, 90% chance it's closed
- Returns either the text "OPEN" or "CLOSED"

Smoke Block:

```python
```

```
elif sensor_type == "smoke":
    if random.random() < 0.05:
        return "DETECTED"
    else:
        return "CLEAR"
```

Explanation:

- 5% chance of smoke (very rare, as it should be!)
- Returns either "DETECTED" or "CLEAR"

Function 2: process_reading()

This function **decides which alerts to trigger** based on the reading.

```python
```

```python
def process_reading(sensor, reading_value, system_mode):
    """
    Processes a sensor reading and determines if alerts are needed.

    Parameters:
    - sensor: Sensor dictionary
    - reading_value: The reading from the sensor
    - system_mode: Current system mode (HOME, AWAY, SLEEP)

    Returns:
    - A list of alerts (empty if no alerts needed)
    """
    alerts = []  # Start with an empty list
    timestamp = datetime.now().strftime("%H:%M:%S")  # Get current time

    # ========== SECURITY ALERTS ==========
    # Check for motion/door triggers while in AWAY mode
    if should_trigger_security_alert(sensor, reading_value, system_mode):
        if sensor["type"] == "motion":
            alert = create_alert(
                "HIGH",
                f"SECURITY: Motion detected in {sensor['location']} while in {system_mode} mode!",
                sensor["id"],
                timestamp
            )
            alerts.append(alert)

        elif sensor["type"] == "door":
            alert = create_alert(
                "HIGH",
                f"SECURITY: {sensor['location']} opened while in {system_mode} mode!",
                sensor["id"],
                timestamp
            )
            alerts.append(alert)

    # ========== SAFETY ALERTS ==========
    # Check for dangerous temperatures
    if sensor["type"] == "temperature":
        if reading_value < 35:
            alert = create_alert(
                "CRITICAL",
                f"SAFETY: Temperature in {sensor['location']} is {reading_value}°F - FROZEN PIPE RISK!",
                sensor["id"],
                timestamp
            )
```

```
            alerts.append(alert)

        elif reading_value > 95:
            alert = create_alert(
                "CRITICAL",
                f"SAFETY: Temperature in {sensor['location']} is {reading_value}°F - EQUIPMENT FAILURE RISK!",
                sensor["id"],
                timestamp
            )
            alerts.append(alert)

    # Check for smoke
    if sensor["type"] == "smoke":
        if reading_value == "DETECTED":
            alert = create_alert(
                "CRITICAL",
                f"SAFETY: SMOKE DETECTED in {sensor['location']} - FIRE RISK!",
                sensor["id"],
                timestamp
            )
            alerts.append(alert)

    # ========== COMFORT NOTIFICATIONS ==========
    # Only in HOME mode, check if temperature is uncomfortable
    if system_mode == "HOME" and sensor["type"] == "temperature":
        if reading_value < 65 or reading_value > 75:
            alert = create_alert(
                "LOW",
                f"COMFORT: Temperature in {sensor['location']} is {reading_value}°F (comfort range is 65-75°F)",
                sensor["id"],
                timestamp
            )
            alerts.append(alert)

    return alerts  # Return the list of alerts (could be empty)
```

Step-by-Step Breakdown:

Line 1: `alerts = []`

- Create an empty list to store alerts we find
- As we check conditions, we'll add alerts to this list

Line 2: `timestamp = datetime.now().strftime("%H:%M:%S")`

- Get the current time

- `.strftime()` converts it to a readable format like "14:30:00"
- HH = hours, MM = minutes, SS = seconds

Security Alerts Block:

```python
if should_trigger_security_alert(sensor, reading_value, system_mode):
    # Create an alert
    alert = create_alert(...)
    # Add it to the list
    alerts.append(alert)
```

Explanation:

- Use the function we created earlier to check for security issues
- If true, create an alert dictionary with:
  - Severity: "HIGH" (serious but not critical)
  - Message: describe what happened (e.g., "Motion detected in Living Room")
  - Sensor ID: which sensor triggered it
  - Timestamp: when it happened
- `alerts.append(alert)` adds this alert to our list

Safety Alerts Block:

```python
if sensor["type"] == "temperature":
    if reading_value < 35:
        alert = create_alert("CRITICAL", ...)
        alerts.append(alert)
    elif reading_value > 95:
        alert = create_alert("CRITICAL", ...)
        alerts.append(alert)
```

Explanation:

- If this is a temperature sensor
- If temperature is below 35°F (freezing), trigger a CRITICAL alert
- If temperature is above 95°F (too hot), trigger a CRITICAL alert
- Use `f-strings` to insert the actual values in the message

Smoke Check:

```python
if sensor["type"] == "smoke":
    if reading_value == "DETECTED":
        alert = create_alert("CRITICAL", ...)
        alerts.append(alert)
```

Explanation:

- If smoke is detected, create a CRITICAL alert immediately

Comfort Notifications Block:

```python
if system_mode == "HOME" and sensor["type"] == "temperature":
    if reading_value < 65 or reading_value > 75:
        alert = create_alert("LOW", ...)
        alerts.append(alert)
```

Explanation:

- Only check this if we're in HOME mode AND it's a temperature sensor
- If the temperature is outside the comfort range (65-75°F), create a LOW severity alert
- This is just a notification, not urgent

Final Line:

```python
return alerts
```

Explanation:

- Give back the list of alerts we found
- This could be 0, 1, or multiple alerts

Function 3: trigger_alert()

This function **displays an alert** in a nice formatted way.

```python
```

```python
def trigger_alert(alert):
    """
    Displays an alert to the user with nice formatting and emojis.

    Parameters:
    - alert: Alert dictionary
    """
    severity_symbol = {
        "LOW": "ℹ️",
        "MEDIUM": "⚠️",
        "HIGH": "🚨",
        "CRITICAL": "🔥"
    }

    symbol = severity_symbol.get(alert["severity"], "⚠️")
    print(f"[ALERT!] {symbol} {alert['severity']}: {alert['message']}")
```

Step-by-Step Breakdown:

Lines 1-5: Create a dictionary of symbols

```python
severity_symbol = {
    "LOW": "ℹ️",
    "MEDIUM": "⚠️",
    "HIGH": "🚨",
    "CRITICAL": "🔥"
}
```

Explanation:

- This dictionary maps severity levels to emoji symbols
- `"LOW"` maps to the info emoji ℹ️
- `"CRITICAL"` maps to the fire emoji 🔥

Lines 7-8: Get the symbol

```python
symbol = severity_symbol.get(alert["severity"], "⚠️")
print(f"[ALERT!] {symbol} {alert['severity']}: {alert['message']}")
```

Explanation:

- `severity_symbol.get(alert["severity"], "⚠️")` looks up the severity in the dictionary

- The second parameter `"⚠️ "` is the **default** if the severity isn't found
- Print the alert in a nice format: `[ALERT!] 🔥 CRITICAL: SMOKE DETECTED in Bedroom - FIRE RISK!`

Function 4: log_event()

This function **logs events** for records.

```python
def log_event(message, timestamp=None):
    """
    Logs an event to the console.

    Parameters:
    - message: The message to log
    - timestamp: Optional timestamp (uses current time if not provided)
    """
    if timestamp is None:
        timestamp = datetime.now().strftime("%H:%M:%S")
    print(f"[LOG] [{timestamp}] {message}")
```

Step-by-Step Breakdown:

Line 1: `if timestamp is None:`

- Check if a timestamp was provided
- If not (`None`), generate one

Line 2: `timestamp = datetime.now().strftime("%H:%M:%S")`

- Get the current time and format it

Line 3: `print(f"[LOG] [{timestamp}] {message}")`

- Print a log entry in a standard format

Example Usage:

```python
log_event("System started")
log_event("Sending notification to homeowner...", "14:31:00")
```

Output:

```
[LOG] [14:30:45] System started
[LOG] [14:31:00] Sending notification to homeowner...
```

Checkpoint: Test Your Code

```python
# Test generate_reading
test_sensor = sensors[0]  # Motion sensor
reading = generate_reading(test_sensor)
print(f"Generated reading for {test_sensor['location']}: {reading}")

# Test process_reading
alerts = process_reading(test_sensor, True, "AWAY")
print(f"Number of alerts generated: {len(alerts)}")

if alerts:
    trigger_alert(alerts[0])

# Test log_event
log_event("Test event logged successfully")
```

STEP 5: CREATING CLASSES

What is a Class?

A **class** is like a **blueprint** or **template** for creating objects.

Real-World Analogy

Think of it like making cookies:

- **Cookie Recipe (Class)** = Instructions for how to make a cookie
- **Cookie (Object)** = An actual cookie you made using the recipe
- **Ingredients (Properties)** = The data (id, location, type)
- **Instructions (Methods)** = The functions (read, isAbnormal, reset)

You can use the same recipe to make many cookies!

The Sensor Class

Instead of using dictionaries and functions separately, we'll combine them into a **class**:

```python
```

```python
class Sensor:
    """
    Represents a sensor in the HomeGuard system.
    """

    def __init__(self, sensor_id, location, sensor_type, threshold=None):
        """
        Initializes a new sensor (the constructor).

        This function runs automatically when you create a new Sensor object.

        Parameters:
        - sensor_id: Unique identifier for the sensor
        - location: Where the sensor is located
        - sensor_type: Type of sensor ("motion", "temperature", "door", "smoke")
        - threshold: Optional threshold value for the sensor
        """
        # Store all the information as properties of this object
        self.id = sensor_id            # Use 'self' to make this belong to this specific sensor
        self.location = location
        self.type = sensor_type
        self.threshold = threshold
        self.current_value = None       # No reading yet

    def read(self):
        """
        Generates and stores a new reading for this sensor.

        Returns:
        - The reading value
        """
        # Generate a realistic reading
        if self.type == "temperature":
            self.current_value = random.randint(30, 100)
        elif self.type == "motion":
            self.current_value = random.random() < 0.2
        elif self.type == "door":
            self.current_value = "OPEN" if random.random() < 0.1 else "CLOSED"
        elif self.type == "smoke":
            self.current_value = "DETECTED" if random.random() < 0.05 else "CLEAR"

        return self.current_value

    def isAbnormal(self):
        """
        Checks if the current reading is abnormal.
```

```
        Returns:
        - True if abnormal, False if normal
        """
        if self.type == "temperature":
            if self.current_value is None:
                return False
            return self.current_value < 35 or self.current_value > 95

        elif self.type == "motion":
            return self.current_value == True

        elif self.type == "door":
            return self.current_value == "OPEN"

        elif self.type == "smoke":
            return self.current_value == "DETECTED"

        return False

    def reset(self):
        """
        Resets the sensor's current reading to None.
        """
        self.current_value = None

    def __str__(self):
        """
        Returns a string representation of the sensor.
        This is what prints when you do print(sensor_object)
        """
        status = "No reading" if self.current_value is None else str(self.current_value)
        return f"{self.id} ({self.location}): {status}"
```

Step-by-Step Breakdown:

Line 1: `class Sensor:`

- Define a new class called `Sensor`
- This is the blueprint

Lines 2-24: The `__init__` method (constructor)

```python
def __init__(self, sensor_id, location, sensor_type, threshold=None):
```

Explanation:

- `__init__` is special - it runs automatically when you create a new sensor
- `self` means "this specific sensor object" (not the blueprint, but an actual object)
- Parameters are information you provide when creating the sensor

Lines 10-14: Store the properties

```python
self.id = sensor_id
self.location = location
self.type = sensor_type
self.threshold = threshold
self.current_value = None
```

Explanation:

- `self.id` creates a property called `id` that belongs to this specific sensor
- When you create a sensor, it automatically gets these properties
- `self.current_value = None` means no reading yet

The `read()` Method (Lines 26-40):

```python
def read(self):
    if self.type == "temperature":
        self.current_value = random.randint(30, 100)
    elif self.type == "motion":
        self.current_value = random.random() < 0.2
    # ... more types
    return self.current_value
```

Explanation:

- This method generates a reading for the specific sensor
- It checks `self.type` (this sensor's type)
- It updates `self.current_value` with the new reading
- It returns the reading

The `isAbnormal()` Method (Lines 42-62):

```python
```

```python
def isAbnormal(self):
    if self.type == "temperature":
        if self.current_value is None:
            return False
        return self.current_value < 35 or self.current_value > 95
    # ... more types
    return False
```

Explanation:

- Checks if `self.current_value` (this sensor's reading) is abnormal
- Uses the same logic we wrote before, but now it's part of the object
- Returns True or False

The `reset()` Method (Lines 64-68):

```python
def reset(self):
    self.current_value = None
```

Explanation:

- Simple method that clears the reading
- Sets `self.current_value` back to None

The `__str__()` Method (Lines 70-76):

```python
def __str__(self):
    status = "No reading" if self.current_value is None else str(self.current_value)
    return f"{self.id} ({self.location}): {status}"
```

Explanation:

- This special method runs when you use `print(sensor_object)`
- Creates a nice text representation: "MOTION_001 (Living Room): False"

How to Use the Class

Instead of dictionaries and functions, you now create **sensor objects**:

```python
```

```python
# Create sensor objects
sensor_objects = [
    Sensor("MOTION_001", "Living Room", "motion"),
    Sensor("TEMP_001", "Kitchen", "temperature", threshold=35),
    Sensor("DOOR_001", "Front Door", "door"),
    Sensor("SMOKE_001", "Bedroom", "smoke")
]

# Use the sensors
sensor = sensor_objects[0]  # Get the first sensor
reading = sensor.read()     # Call the read method
print(f"Reading: {reading}")

if sensor.isAbnormal():     # Call the isAbnormal method
    print("This reading is abnormal!")

sensor.reset()              # Clear the reading
print(f"After reset: {sensor.current_value}")  # None
```

Checkpoint: Test Your Class

```python
# Create and test a sensor
test_sensor = Sensor("TEST_001", "Test Room", "temperature", threshold=35)

# Generate a reading
test_sensor.read()
print(f"Sensor reading: {test_sensor.current_value}")

# Check if abnormal
print(f"Is abnormal: {test_sensor.isAbnormal()}")

# Print the sensor info
print(f"Sensor info: {test_sensor}")

# Reset it
test_sensor.reset()
print(f"After reset: {test_sensor.current_value}")
```

STEP 6: INTEGRATION - PUTTING IT ALL TOGETHER

What You're Doing

Creating a **main simulation loop** that runs your entire system.

The run_simulation() Function

```python
```

```python
def run_simulation(duration_minutes=5, system_mode="AWAY"):
    """
    Runs the HomeGuard security system simulation.

    Parameters:
    - duration_minutes: How long to run the simulation (default: 5 minutes)
    - system_mode: System mode (HOME, AWAY, SLEEP)
    """
    print("=" * 50)
    print("=== HomeGuard Security System ===")
    print("=" * 50)
    print(f"Mode: {system_mode}\n")

    # Create sensor objects
    sensors = [
        Sensor("MOTION_001", "Living Room", "motion"),
        Sensor("TEMP_001", "Kitchen", "temperature", threshold=35),
        Sensor("DOOR_001", "Front Door", "door"),
        Sensor("SMOKE_001", "Bedroom", "smoke")
    ]

    # Simulate time passing (each iteration = 1 minute)
    for minute in range(duration_minutes):
        current_time = datetime.now().strftime("%H:%M:%S")
        print(f"\nTime: {current_time}")

        # Read all sensors
        for sensor in sensors:
            reading = sensor.read()  # Generate a new reading

            # Display the reading in a nice format
            if sensor.type == "temperature":
                status = "Normal" if 65 <= reading <= 75 else "Abnormal"
                print(f"[READING] {sensor.location} Temperature: {reading}°F ({status})")

            elif sensor.type == "motion":
                status = "DETECTED" if reading else "No activity"
                print(f"[READING] {sensor.location} Motion: {status}")

            elif sensor.type == "door":
                print(f"[READING] {sensor.location}: {reading}")

            elif sensor.type == "smoke":
                print(f"[READING] {sensor.location} Smoke: {reading}")

            # Process the reading and trigger alerts
```

```python
        # Convert sensor object to dict format for process_reading function
        sensor_dict = {
            "id": sensor.id,
            "location": sensor.location,
            "type": sensor.type,
            "threshold": sensor.threshold
        }
        alerts = process_reading(sensor_dict, reading, system_mode)

        # Display all alerts
        for alert in alerts:
            trigger_alert(alert)
            # If it's serious, log that we're sending a notification
            if alert["severity"] in ["HIGH", "CRITICAL"]:
                log_event("Sending notification to homeowner...")

    # Small delay to make output readable (optional)
    import time
    time.sleep(0.5)  # Wait 0.5 seconds before next iteration

# Simulation complete
print("\n" + "=" * 50)
print("Simulation complete!")
print("=" * 50)
```

Step-by-Step Breakdown:

Setup Section (Lines 1-11):

```python
print("=" * 50)
print("=== HomeGuard Security System ===")
print("=" * 50)
print(f"Mode: {system_mode}\n")
```

Explanation:

- `"=" * 50` creates a line of 50 equal signs
- Print the title and current mode

Sensor Creation (Lines 13-19):

```python
```

```
sensors = [
    Sensor("MOTION_001", "Living Room", "motion"),
    Sensor("TEMP_001", "Kitchen", "temperature", threshold=35),
    Sensor("DOOR_001", "Front Door", "door"),
    Sensor("SMOKE_001", "Bedroom", "smoke")
]
```

Explanation:

- Create a list of 4 Sensor objects
- Each one is created using the Sensor class we defined

Main Loop (Line 22):

```python
for minute in range(duration_minutes):
```

Explanation:

- Loop a certain number of times (default 5)
- Each iteration represents 1 minute of simulation
- `range(5)` means: 0, 1, 2, 3, 4 (5 iterations)

Get Current Time (Lines 23-24):

```python
current_time = datetime.now().strftime("%H:%M:%S")
print(f"\nTime: {current_time}")
```

Explanation:

- Get the current time and format it
- Print the time for this iteration

Sensor Reading Loop (Lines 26-43):

```python
for sensor in sensors:
    reading = sensor.read()  # Generate a new reading
```

Explanation:

- Loop through each sensor

- Call `sensor.read()` to generate a reading
- Store it in the `reading` variable

Display the Reading (Lines 45-55):

```python
if sensor.type == "temperature":
    status = "Normal" if 65 <= reading <= 75 else "Abnormal"
    print(f"[READING] {sensor.location} Temperature: {reading}°F ({status})")
```

Explanation:

- Check what type of sensor this is
- Format the output appropriately
- For temperature, show if it's in the normal comfort range (65-75°F)
- Print in a clear format: `[READING] Kitchen Temperature: 68°F (Normal)`

Process the Reading (Lines 57-63):

```python
sensor_dict = {
    "id": sensor.id,
    "location": sensor.location,
    "type": sensor.type,
    "threshold": sensor.threshold
}
alerts = process_reading(sensor_dict, reading, system_mode)
```

Explanation:

- We have a Sensor object, but `process_reading()` expects a dictionary
- Create a temporary dictionary with the sensor information
- Call `process_reading()` to check for alerts
- Get back a list of alerts (could be empty)

Display Alerts (Lines 65-69):

```python
for alert in alerts:
    trigger_alert(alert)
    if alert["severity"] in ["HIGH", "CRITICAL"]:
        log_event("Sending notification to homeowner...")
```

Explanation:

- Loop through each alert
- Display it using `trigger_alert()`
- If it's serious (HIGH or CRITICAL), also log that we're sending a notification

Delay (Lines 71-72):

```python
import time
time.sleep(0.5)
```

Explanation:

- `time.sleep(0.5)` pauses the program for 0.5 seconds
- This makes the output easier to read

Final Message (Lines 74-76):

```python
print("\n" + "=" * 50)
print("Simulation complete!")
print("=" * 50)
```

Explanation:

- After all iterations are done, print a completion message

Running the Simulation

At the bottom of your code, add:

```python
# Main execution
if __name__ == "__main__":
    # Run the simulation with AWAY mode for 3 minutes
    run_simulation(duration_minutes=3, system_mode="AWAY")
```

What Does `if __name__ == "__main__":` Mean?

This checks if you're running the file directly (not importing it).

- If you run it directly: `True` - the code inside runs
- If another script imports this: `False` - the code doesn't run

This is a Python convention.

Checkpoint: Run Your Complete Code

Run your entire notebook. You should see output like:

```
==================================================
=== HomeGuard Security System ===
==================================================
Mode: AWAY

Time: 14:30:00
[READING] Living Room Motion: No activity
[READING] Kitchen Temperature: 68°F (Normal)
[READING] Front Door: CLOSED
[READING] Bedroom Smoke: CLEAR

Time: 14:31:00
[READING] Living Room Motion: DETECTED
[ALERT!] 🚨 HIGH: SECURITY: Motion detected in Living Room while in AWAY mode!
[LOG] [14:31:00] Sending notification to homeowner...
[READING] Kitchen Temperature: 34°F (Normal)
[ALERT!] 🔥 CRITICAL: SAFETY: Temperature in Kitchen is 34°F - FROZEN PIPE RISK!
[LOG] [14:31:00] Sending notification to homeowner...
...
==================================================
Simulation complete!
==================================================
```

STEP 7: TESTING YOUR SYSTEM

What You're Doing

Running the simulator in different scenarios to verify everything works correctly.

Test 1: AWAY Mode Security Test

```python
print("\n\n=== TEST 1: AWAY MODE SECURITY ===")
run_simulation(duration_minutes=2, system_mode="AWAY")
```

What to Look For:

- Motion and door alerts should appear when system is in AWAY mode
- Alerts should have HIGH severity
- Output should show "SECURITY:" in the alert message

Test 2: HOME Mode Comfort Test

```python
print("\n\n=== TEST 2: HOME MODE COMFORT ===")
run_simulation(duration_minutes=2, system_mode="HOME")
```

What to Look For:

- Motion and door alerts should NOT appear (because you're home)
- Temperature alerts outside 65-75°F should show as LOW severity comfort notifications
- Smoke alerts should still appear (safety trumps comfort)

Test 3: SLEEP Mode Test

```python
print("\n\n=== TEST 3: SLEEP MODE ===")
run_simulation(duration_minutes=2, system_mode="SLEEP")
```

What to Look For:

- System should run without security alerts for normal motion/door activity
- Safety alerts (temperature, smoke) should still trigger

Expected Output Format

Your output should look like this:

```
==================================================
=== HomeGuard Security System ===
==================================================
Mode: AWAY

Time: 14:30:00

[READING] Living Room Motion: No activity
[READING] Kitchen Temperature: 72°F (Normal)
[READING] Front Door: CLOSED
[READING] Bedroom Smoke: CLEAR

Time: 14:31:00

[READING] Living Room Motion: No activity
[READING] Kitchen Temperature: 68°F (Normal)
[READING] Front Door: OPENED
[ALERT!] 🚨 HIGH: SECURITY: Front Door opened while in AWAY mode!
[LOG] [14:31:00] Sending notification to homeowner...
[READING] Bedroom Smoke: CLEAR

==================================================
Simulation complete!
==================================================
```

What Each Alert Level Means

- 🔥 **CRITICAL**: Extreme danger! Fire, extreme temperature, etc.
- 🚨 **HIGH**: Security threat! Intrusion detected.
- ⚠️ **MEDIUM**: Moderate issue. (Not used in this basic version)
- ℹ️ **LOW**: Minor notification. Temperature slightly outside comfort range.

Troubleshooting Guide

Problem: "NameError: name 'sensor' is not defined"

Solution: Make sure you created your sensors list before trying to use it. Check that all variable names match exactly (Python is case-sensitive).

Problem: "TypeError: '>' not supported between instances of 'str' and 'int'"

Solution: You're comparing different data types. Make sure:

- Temperature comparisons use numbers: `reading_value < 35` ✓
- Door comparisons use text: `reading_value == "OPEN"` ✓

- Motion comparisons use booleans: `reading_value == True` ✓

Problem: "Alerts not triggering when they should"

Solution: Check your if/else conditions. Remember:

- `==` is for **comparing** (checking if equal)
- `=` is for **assigning** (storing a value)

```python
if system_mode == "AWAY":  # Correct - comparing with ==
    system_mode = "AWAY"   # Correct - assigning with =
```

Problem: "Random values not generating correctly"

Solution: Make sure you imported random at the top:

```python
import random
```

Problem: "Class methods not working"

Solution: Remember:

- Methods must include `self` as first parameter
- Call methods on objects: `sensor.read()` ✓
- Don't call on class itself: `Sensor.read()` ✗

Problem: "Output formatting looks wrong"

Solution: Check your f-strings:

```python
f"Temperature: {value}°F"  # Correct - variable in curly braces
f"Temperature: value°F"    # Wrong - variable not in curly braces
```

COMPLETE CODE REFERENCE

Here's the ENTIRE program in one place for easy copying:

```python
```

```python
"""
HomeGuard Security System Simulator
Author: Your Name
Description: A smart home monitoring system that processes sensor readings
             and triggers alerts for security, safety, and comfort issues.
"""

import random
from datetime import datetime
import time

# System configuration
HOME_MODES = ["HOME", "AWAY", "SLEEP"]
ALERT_SEVERITIES = ["LOW", "MEDIUM", "HIGH", "CRITICAL"]

# Current system state
current_mode = "AWAY"

# ================================================================
# STEP 2: DATA STRUCTURES (Dictionaries and Functions)
# ================================================================

def create_sensor(sensor_id, location, sensor_type, threshold=None):
    """Creates a sensor data structure."""
    sensor = {
        "id": sensor_id,
        "location": location,
        "type": sensor_type,
        "current_value": None,
        "threshold": threshold
    }
    return sensor

def create_alert(severity, message, sensor_id, timestamp):
    """Creates an alert data structure."""
    alert = {
        "severity": severity,
        "message": message,
        "sensor_id": sensor_id,
        "timestamp": timestamp
    }
    return alert

# ================================================================
# STEP 3: IF/ELSE LOGIC
# ================================================================
```

```python
def is_abnormal_reading(sensor, reading_value):
    """Checks if a sensor reading is abnormal."""
    sensor_type = sensor["type"]

    if sensor_type == "temperature":
        if reading_value < 35 or reading_value > 95:
            return True
        else:
            return False

    elif sensor_type == "motion":
        if reading_value == True:
            return True
        else:
            return False

    elif sensor_type == "door":
        if reading_value == "OPEN":
            return True
        else:
            return False

    elif sensor_type == "smoke":
        if reading_value == "DETECTED":
            return True
        else:
            return False

    return False

def should_trigger_security_alert(sensor, reading_value, system_mode):
    """Determines if a security alert should be triggered."""
    sensor_type = sensor["type"]

    if sensor_type == "motion" or sensor_type == "door":
        if system_mode == "AWAY":
            if is_abnormal_reading(sensor, reading_value):
                return True
            else:
                return False
        else:
            return False
    else:
        return False

# ==============================================================
```

```python
# STEP 4: FUNCTIONS FOR PROCESSING
# ================================================================

def generate_reading(sensor):
    """Generates a realistic reading for a sensor."""
    sensor_type = sensor["type"]

    if sensor_type == "temperature":
        return random.randint(30, 100)

    elif sensor_type == "motion":
        return random.random() < 0.2

    elif sensor_type == "door":
        if random.random() < 0.1:
            return "OPEN"
        else:
            return "CLOSED"

    elif sensor_type == "smoke":
        if random.random() < 0.05:
            return "DETECTED"
        else:
            return "CLEAR"

    return None

def process_reading(sensor, reading_value, system_mode):
    """Processes a sensor reading and determines if alerts are needed."""
    alerts = []
    timestamp = datetime.now().strftime("%H:%M:%S")

    # ========== SECURITY ALERTS ==========
    if should_trigger_security_alert(sensor, reading_value, system_mode):
        if sensor["type"] == "motion":
            alert = create_alert(
                "HIGH",
                f"SECURITY: Motion detected in {sensor['location']} while in {system_mode} mode!",
                sensor["id"],
                timestamp
            )
            alerts.append(alert)

        elif sensor["type"] == "door":
            alert = create_alert(
                "HIGH",
                f"SECURITY: {sensor['location']} opened while in {system_mode} mode!",
```

```
            sensor["id"],
            timestamp
        )
        alerts.append(alert)

# ========== SAFETY ALERTS ==========
if sensor["type"] == "temperature":
    if reading_value < 35:
        alert = create_alert(
            "CRITICAL",
            f"SAFETY: Temperature in {sensor['location']} is {reading_value}°F - FROZEN PIPE RISK!",
            sensor["id"],
            timestamp
        )
        alerts.append(alert)

    elif reading_value > 95:
        alert = create_alert(
            "CRITICAL",
            f"SAFETY: Temperature in {sensor['location']} is {reading_value}°F - EQUIPMENT FAILURE RISK!",
            sensor["id"],
            timestamp
        )
        alerts.append(alert)

if sensor["type"] == "smoke":
    if reading_value == "DETECTED":
        alert = create_alert(
            "CRITICAL",
            f"SAFETY: SMOKE DETECTED in {sensor['location']} - FIRE RISK!",
            sensor["id"],
            timestamp
        )
        alerts.append(alert)

# ========== COMFORT NOTIFICATIONS ==========
if system_mode == "HOME" and sensor["type"] == "temperature":
    if reading_value < 65 or reading_value > 75:
        alert = create_alert(
            "LOW",
            f"COMFORT: Temperature in {sensor['location']} is {reading_value}°F (comfort range is 65-75°F)",
            sensor["id"],
            timestamp
        )
        alerts.append(alert)

return alerts
```

```python
def trigger_alert(alert):
    """Displays an alert to the user."""
    severity_symbol = {
        "LOW": "ℹ️ ",
        "MEDIUM": "⚠️ ",
        "HIGH": "🚨 ",
        "CRITICAL": "🔥 "
    }

    symbol = severity_symbol.get(alert["severity"], "⚠️ ")
    print(f"[ALERT!] {symbol} {alert['severity']}: {alert['message']}")

def log_event(message, timestamp=None):
    """Logs an event to the console."""
    if timestamp is None:
        timestamp = datetime.now().strftime("%H:%M:%S")
    print(f"[LOG] [{timestamp}] {message}")

# ==================================================================
# STEP 5: THE SENSOR CLASS
# ==================================================================

class Sensor:
    """Represents a sensor in the HomeGuard system."""

    def __init__(self, sensor_id, location, sensor_type, threshold=None):
        """Initializes a new sensor."""
        self.id = sensor_id
        self.location = location
        self.type = sensor_type
        self.threshold = threshold
        self.current_value = None

    def read(self):
        """Generates and stores a new reading for this sensor."""
        if self.type == "temperature":
            self.current_value = random.randint(30, 100)
        elif self.type == "motion":
            self.current_value = random.random() < 0.2
        elif self.type == "door":
            self.current_value = "OPEN" if random.random() < 0.1 else "CLOSED"
        elif self.type == "smoke":
            self.current_value = "DETECTED" if random.random() < 0.05 else "CLEAR"

        return self.current_value
```

```python
    def isAbnormal(self):
        """Checks if the current reading is abnormal."""
        if self.type == "temperature":
            if self.current_value is None:
                return False
            return self.current_value < 35 or self.current_value > 95

        elif self.type == "motion":
            return self.current_value == True

        elif self.type == "door":
            return self.current_value == "OPEN"

        elif self.type == "smoke":
            return self.current_value == "DETECTED"

        return False

    def reset(self):
        """Resets the sensor's current reading to None."""
        self.current_value = None

    def __str__(self):
        """Returns a string representation of the sensor."""
        status = "No reading" if self.current_value is None else str(self.current_value)
        return f"{self.id} ({self.location}): {status}"

# ================================================================
# STEP 6: INTEGRATION - THE MAIN SIMULATION
# ================================================================

def run_simulation(duration_minutes=5, system_mode="AWAY"):
    """Runs the HomeGuard security system simulation."""
    print("=" * 50)
    print("=== HomeGuard Security System ===")
    print("=" * 50)
    print(f"Mode: {system_mode}\n")

    # Create sensor objects
    sensors = [
        Sensor("MOTION_001", "Living Room", "motion"),
        Sensor("TEMP_001", "Kitchen", "temperature", threshold=35),
        Sensor("DOOR_001", "Front Door", "door"),
        Sensor("SMOKE_001", "Bedroom", "smoke")
    ]

    # Simulate time passing
```

```
    for minute in range(duration_minutes):
        current_time = datetime.now().strftime("%H:%M:%S")
        print(f"\nTime: {current_time}")

        # Read all sensors
        for sensor in sensors:
            reading = sensor.read()

            # Display the reading
            if sensor.type == "temperature":
                status = "Normal" if 65 <= reading <= 75 else "Abnormal"
                print(f"[READING] {sensor.location} Temperature: {reading}°F ({status})")
            elif sensor.type == "motion":
                status = "DETECTED" if reading else "No activity"
                print(f"[READING] {sensor.location} Motion: {status}")
            elif sensor.type == "door":
                print(f"[READING] {sensor.location}: {reading}")
            elif sensor.type == "smoke":
                print(f"[READING] {sensor.location} Smoke: {reading}")

            # Process the reading
            sensor_dict = {
                "id": sensor.id,
                "location": sensor.location,
                "type": sensor.type,
                "threshold": sensor.threshold
            }
            alerts = process_reading(sensor_dict, reading, system_mode)

            # Trigger alerts
            for alert in alerts:
                trigger_alert(alert)
                if alert["severity"] in ["HIGH", "CRITICAL"]:
                    log_event("Sending notification to homeowner...")

        time.sleep(0.5)

    print("\n" + "=" * 50)
    print("Simulation complete!")
    print("=" * 50)

# ================================================================
# STEP 7: MAIN EXECUTION
# ================================================================

if __name__ == "__main__":
    # Run different tests
```

```
print("\n=== SECURITY TEST: AWAY MODE ===")
run_simulation(duration_minutes=3, system_mode="AWAY")

print("\n\n=== COMFORT TEST: HOME MODE ===")
run_simulation(duration_minutes=3, system_mode="HOME")

print("\n\n=== SAFETY TEST: ALL MODES ===")
run_simulation(duration_minutes=2, system_mode="SLEEP")
```

KEY TAKEAWAYS

Concepts You Learned:

1. **Dictionaries**: Store related information with labels (keys and values)
2. **Functions**: Reusable blocks of code that take input and return output
3. **If/Else**: Decision making in code (checking conditions)
4. **Lists**: Collections of items (dictionaries, objects, etc.)
5. **Classes**: Templates for creating objects with properties and methods
6. **Objects**: Instances of a class that have their own data and methods
7. **For Loops**: Repeating code for each item in a collection
8. **Imports**: Bringing in external Python tools (random, datetime)

Why This Matters:

This lab teaches you patterns used in **real AI systems**:

- **Data Structures**: How AI systems organize sensor data
- **Decision Making**: How AI systems make choices based on data
- **Automation**: How systems can run continuously and respond to events
- **Object-Oriented Programming**: How complex systems are built from reusable pieces

Next Steps:

- Add more sensor types
- Implement false-alarm detection (require multiple confirmations)
- Create a daily summary report
- Add machine learning to detect patterns over time

You did it! You built a complete smart home monitoring system! 🎉